6901012610196 Chonpatchut Boonpalit Cpr.E S.2

รายงานผลการปฏิบัติการ

Graphic Mode

Exercise: Positive

สิ่งที่ทำ

- ฝึกใช้การเปรียบเทียบค่าร่วมกับเงื่อนไข เพื่อกำหนดผลลัพธ์ที่จะแสดงบนหน้าจอกราฟิก
- ใช้คำสั่ง `if` ในการเลือกการทำงานตามค่าที่รับเข้ามา

Pseudo Code

1. รับค่าที่ต้องการตรวจสอบ
2. เปรียบเทียบค่ากับเงื่อนไข
3. หากเป็นจริงให้แสดงผลแบบหนึ่ง
4. หากเป็นเท็จให้แสดงผลอีกแบบ

อธิบายโค้ด

- รับข้อมูลจากผู้ใช้
- ใช้ตัวดำเนินการเปรียบเทียบ เช่น `>`, `<`, `==`
- ใช้ `if` ตรวจสอบเงื่อนไข
- แสดงผลตามเงื่อนไขที่เป็นจริง

Error ที่พบ

- ใช้เครื่องหมาย `=` แทน `==`
- เยื้องบรรทัด (Indentation) ไม่ถูกต้อง
- เปรียบเทียบชนิดข้อมูลไม่ตรงกัน

Exercise: Interaction

สิ่งที่ทำ

- ฝึกสร้างโปรแกรมที่ตอบสนองต่อการโต้ตอบของผู้ใช้
- ใช้เงื่อนไขเพื่อเปลี่ยนผลลัพธ์ตามข้อมูลที่ได้รับ

Pseudo Code

1. รับข้อมูลจากผู้ใช้
2. ตรวจสอบเงื่อนไข

3. เปลี่ยนการแสดงผลตามค่าที่รับ

อธิบายโค้ด

- รับค่าจากเมาส์หรือคีย์บอร์ด
- เปรียบเทียบข้อมูล
- ใช้คำสั่งเงื่อนไขเลือกการทำงาน
- แสดงผลบนหน้าจอ

Error ที่พบ

- รับค่าผิดชนิดข้อมูล
- ลืมกำหนดตัวแปรก่อนใช้งาน
- เงื่อนไขซ้อนกันผิดลำดับ

Text Mode

โปรแกรม BMI

สิ่งที่ทำ

- คำนวณค่า BMI
- แยกการคำนวณระหว่างเด็กและผู้ใหญ่
- แสดงสถานะของน้ำหนักตามช่วง BMI

Pseudo Code

1. รับอายุ
2. ถ้าอายุ 2–19 ปี เรียกโปรแกรมเด็ก
3. ถ้าอายุตั้งแต่ 20 ปีขึ้นไป เรียกโปรแกรมผู้ใหญ่
4. รับน้ำหนักและส่วนสูง
5. คำนวณ BMI
6. เปรียบเทียบกับเกณฑ์
7. แสดงผล

อธิบายโค้ดทีละส่วน

```
from datetime import datetime
```

- นำเข้าโมดูลสำหรับคำนวณอายุ

```
calculate_bmi(weight,height)
```

- รับน้ำหนักและส่วนสูง

- ส่งค่ากลับ BMI ด้วย `return weight/(height**2)`

`adult_bmi_status(bmi)`

- ใช้ `if-elif-else`
- เปรียบเทียบ BMI ด้วย `<`
- ส่งข้อความสถานะกลับด้วย `return`

`calculate_age(birth_date)`

- คำนวณปี เดือน วัน
- ส่งค่ากลับพร้อมกัน 3 ค่า

`calculate_child_percentile(...)`

- ฟังก์ชันตัวอย่าง
- ส่งค่า Percentile กลับด้วย `return`

`child_bmi_status(percentile,bmi)`

- ใช้เงื่อนไขหลายระดับ
- มี Nested if ภายใน `elif`
- ส่งผลลัพธ์กลับด้วย `return`

`adult_program()`

- รับน้ำหนักและส่วนสูง
- เรียก `calculate_bmi()`
- เรียก `adult_bmi_status()`
- แสดงผล

`child_program()`

- รับเพศ วันเกิด น้ำหนัก ส่วนสูง
- เรียกฟังก์ชันคำนวณอายุ
- เรียกฟังก์ชันคำนวณ BMI
- เรียกฟังก์ชันหาเปอร์เซ็นไทล์
- แสดงผล

`main()`

- รับอายุ
- ใช้ `if-elif-else`
- เลือกเรียก Child Program หรือ Adult Program

Error ที่พบ

- ลืมแปลงข้อมูลเป็น `float()`
- หารด้วยศูนย์เมื่อส่วนสูงเป็น 0
- ใส่วันที่ผิดทำให้ `datetime()` เกิด Error
- ป้อนตัวอักษรแทนตัวเลขเกิด `ValueError`

โปรแกรม Grade

สิ่งที่ทำ

- รับคะแนน
- เปรียบเทียบช่วงคะแนน
- คืนค่าเกรด

Pseudo Code

1. รับคะแนน
2. ตรวจสอบช่วงคะแนน
3. ส่งเกรดกลับ
4. แสดงผล

อธิบายโค้ด

```
score=float(input(...))
```

- รับคะแนน

```
grade_calculator(score)
```

- ใช้ `if-elif-else`
- เปรียบเทียบด้วย `>=` และ `<`
- คืนค่าเกรดด้วย `return`

```
print(...)
```

- แสดงผลเกรดที่ได้รับ

Error ที่พบ

- คะแนนเกิน 100
- คะแนนติดลบ
- ป้อนตัวอักษรแทนตัวเลข

โปรแกรม Area

สิ่งที่ทำ

- คำนวณพื้นที่
- ตรวจสอบว่าพื้นที่เพียงพอสำหรับกีฬาใด

Pseudo Code

1. รับกว้างและยาว
2. คำนวณพื้นที่
3. เปรียบเทียบกับพื้นที่มาตรฐาน
4. แสดงกีฬาที่สามารถเล่นได้

อธิบายโค้ด

`calculate_area()`

- คูณกว้างกับยาว
- ส่งค่าพื้นที่กลับ

`can_play_sport()`

- เปรียบเทียบ `area >= sport_area`
- ส่งค่า Boolean กลับ

รับค่าความกว้างและความยาว

- แปลงเป็น `float`

กำหนดพื้นที่มาตรฐานกีฬา

- Football
- Basketball
- Volleyball

`if can_play_sport(...)`

- ถ้าพื้นที่พอให้แสดงชื่อกีฬา

`not (A or B or C)`

- ใช้ `or` ตรวจสอบหลายเงื่อนไข
- ใช้ `not` กลับค่าผลลัพธ์
- ถ้าไม่เข้าเงื่อนไขใดเลย แสดงว่าไม่มีสนามมาตรฐาน

Error ที่พบ

- รับค่าติดลบ
- ลืมคืนค่าในฟังก์ชัน
- เรียกฟังก์ชันโดยส่งจำนวน Argument ไม่ครบ

โปรแกรม Ball

สิ่งที่ทำ

- เปรียบเทียบจำนวนลูกกีฬา
- แสดงว่าชนิดใดมีจำนวนมากที่สุด

Pseudo Code

1. รับจำนวนลูกฟุตบอล
2. รับจำนวนลูกบาส
3. รับจำนวนลูกวอลเลย์
4. เปรียบเทียบ
5. แสดงผล

อธิบายโค้ด

รับข้อมูลทั้งสามค่า

- ใช้ `int(input())`

```
if football > basketball and football > volleyball
```

- ใช้ `and`
- ตรวจสอบว่าฟุตบอลมากที่สุด

```
elif
```

- ตรวจสอบบาสมากที่สุด

```
elif
```

- ตรวจสอบวอลเลย์มากที่สุด

```
else
```

- กรณีจำนวนเท่ากัน

Error ที่พบ

- ป้อนตัวอักษรแทนจำนวนเต็ม
- ลืมใช้ `and`

- ใช้ `>` แทน `>=` ในกรณีที่ต้องการรองรับค่าเท่ากัน

โปรแกรม Electricity Bill (Extra)

สิ่งที่ทำ

- คำนวณค่าไฟ
- คิดส่วนลด
- จัดระดับการใช้ไฟ

Pseudo Code

1. รับจำนวนหน่วยไฟ
2. เลือกอัตราค่าไฟ
3. คำนวณค่าไฟ
4. ตรวจสอบส่วนลด
5. คำนวณยอดสุทธิ
6. แสดงผล
7. แสดงระดับการใช้ไฟ

อธิบายโค้ด

`calculate_bill(unit)`

- ใช้ `if-elif-else` เลือกอัตราค่าไฟ
- คำนวณค่าไฟ
- ใช้ Nested if ตรวจสอบส่วนลด
- ส่งค่า Rate, Discount และ Total กลับด้วย `return`

`try`

- ป้องกันการรับข้อมูลผิดชนิด

`if unit < 0`

- ตรวจสอบค่าติดลบ

เรียก `calculate_bill()`

- รับค่าที่ส่งกลับ

แสดงผลใบแจ้งค่าไฟ

ตรวจสอบระดับการใช้ไฟ

- ใช้ `and`

- ใช้ `or`
- ใช้ `not`

`except ValueError`

- จัดการเมื่อป้อนข้อมูลที่ไม่ใช่ตัวเลข

Error ที่พบ

- ป้อนข้อความแทนตัวเลขเกิด `ValueError`
- หน่วยไฟติดลบ
- ลืม `return` ทำให้รับค่ากลับไม่ได้
- ลำดับเงื่อนไขผิดทำให้เลือกอัตราค่าไฟไม่ถูกต้อง

สรุป

จากการทดลองได้ใช้ตัวดำเนินการเปรียบเทียบ ได้แก่ `>`, `<`, `>=`, `<=`, `==` และ `!=` ร่วมกับตัวดำเนินการตรรกะ `and`, `or` และ `not` เพื่อสร้างเงื่อนไขในการตัดสินใจของโปรแกรม ทั้งรูปแบบ `if`, `if-else`, `if-elif-else` และเงื่อนไขแบบซ้อน (Nested if) นอกจากนี้ยังมีการสร้างฟังก์ชันที่ส่งค่ากลับด้วย `return` เพื่อนำผลลัพธ์ไปใช้งานต่อ ทำให้โค้ดแบ่งเป็นส่วนย่อยที่อ่านง่ายและนำกลับมาใช้ซ้ำได้ พร้อมทั้งได้ฝึกแก้ไขข้อผิดพลาดพื้นฐาน เช่น `ValueError`, การเยื้องบรรทัดผิด, การใช้ตัวดำเนินการเปรียบเทียบผิด และการส่งค่ากลับจากฟังก์ชันไม่ถูกต้อง